

宝宝，这份 slide 的主线其实很清楚：

CPU 调度 = 当有多个 runnable 进程时，操作系统决定“下一刻让谁上 CPU”。

中断/异常提供“切换的机会”，调度算法决定“切给谁”。slide 里也明确区分了：中断/异常是进程切换的方法，调度是进程切换的决策。

下面我按重点讲。

1. CPU 调度到底在优化什么？

CPU 调度有几个常见目标，最重要的是这三个：

1. turnaround time 周转时间
2. response time 响应时间
3. fairness 公平性

1.1 Turnaround time：从到达完成

公式是：

$$\text{turnaround time} = \text{completion time} - \text{arrival time}$$

也就是一个任务从来到系统，到彻底完成，总共花了多久。

例如任务 A 在 $t=0$ 到达， $t=10$ 完成：

$$\text{turnaround time} = 10 - 0 = 10$$

这个指标看的是：

任务整体完成得快不快。

批处理任务、大计算任务、编译任务通常比较关心它。

1.2 Response time : 从到达第一次运行

公式是：

$$\text{response time} = \text{first run time} - \text{arrival time}$$

也就是用户提交任务后，多久第一次得到系统响应。

例如任务 A 在 $t=0$ 到达， $t=3$ 第一次上 CPU， $t=10$ 完成：

$$\text{response time} = 3 - 0 = 3$$

$$\text{turnaround time} = 10 - 0 = 10$$

这个指标看的是：

系统理不理我快不快。

交互式程序，比如 shell、编辑器、GUI、服务器请求，更关心 response time。slide 里也强调，SJF 这类算法可能让 response time 很差。

1.3 Fairness : 公平性

公平性不是只看“平均值好不好”，而是看每个任务是否都能获得合理 CPU 资源。

比如平均 turnaround time 可能很好，但是某个长任务一直被短任务插队，迟迟不运行，这就是不公平，严重时叫：

starvation 饥饿

slide 里说得很关键：平均 turnaround time 反映整体性能，但可能牺牲某些 job；如果不断有新 job 到达，某些 job 可能一直不被调度。效率和公平往往不能兼得。

2. 基本调度算法

2.1 FIFO / FCFS : 先来先服务

FIFO , 也叫 FCFS :

First In, First Out

First Come, First Served

谁先到 , 谁先运行。

例如 A、B、C 同时到达 , 运行时间都是 10 :

A: 0 ~ 10

B: 10 ~ 20

C: 20 ~ 30

周转时间 :

A = 10

B = 20

C = 30

平均 turnaround time = $(10 + 20 + 30) / 3 = 20$

优点 :

简单

容易实现

没有复杂判断

缺点是 : 长任务排在前面会拖死后面的短任务。

例如 :

A = 100

B = 10

C = 10

如果 FIFO 顺序是 A、B、C：

A 完成时间 = 100

B 完成时间 = 110

C 完成时间 = 120

平均 turnaround time = $(100 + 110 + 120) / 3 = 110$

这就非常差。slide 里正是用这个例子说明 FIFO 在任务长度差距大时有明显缺点。

2.2 SJF：最短作业优先

SJF，全称：

Shortest Job First

意思是：

每次选择运行时间最短的任务先跑。

对于刚才的例子：

A = 100

B = 10

C = 10

SJF 会先跑 B、C，再跑 A：

B: 0 ~ 10

C: 10 ~ 20

A: 20 ~ 120

周转时间：

B = 10

C = 20

A = 120

平均 turnaround time = $(10 + 20 + 120) / 3 = 50$

比 FIFO 的 110 好很多。

所以 SJF 的核心优点是：

可以显著降低平均 turnaround time

而且有个重要结论：

如果所有任务同时到达，并且我们知道每个任务的运行时间，那么 SJF 对平均 turnaround time 是最优的。

但是它有两个大问题。

第一个问题：**你通常不知道任务到底要运行多久。**

现实中进程未来要跑多久，操作系统一般不可能精确预知。这也是 slide 后面引出 MLFQ 的原因。

第二个问题：**如果长任务先到，SJF 也救不了。**

比如：

A 在 $t=0$ 到达，运行 100
B 在 $t=10$ 到达，运行 10
C 在 $t=10$ 到达，运行 10

如果 A 已经开始跑了，而 SJF 是非抢占式的，那 A 会一直跑完。

于是 B、C 只能等到 A 完成后再跑：

A: 0 ~ 100
B: 100 ~ 110
C: 110 ~ 120

这种情况下平均 turnaround time 仍然很差。slide 里也专门指出：长任务 A 先到达时，SJF 没办法。

2.3 STCF / PSJF：剩余时间最短优先

为了解决“长任务已经开始跑，短任务后来才到”的问题，引入抢占。

STCF :

Shortest Time-to-Completion First

也叫 :

Preemptive Shortest Job First, PSJF

它的规则是 :

每次调度时, 选择“剩余运行时间最短”的任务。

它和 SJF 的区别是 :

SJF : 看总运行时间, 通常不抢占

STCF : 看剩余时间, 允许抢占

还是刚才例子 :

A: $t=0$ 到达, 需要 100

B: $t=10$ 到达, 需要 10

C: $t=10$ 到达, 需要 10

执行过程 :

$t=0 \sim 10$: A 运行, A 剩余 90

$t=10$: B、C 到达, 它们只需要 10, 比 A 剩余的 90 短

于是抢占 A

$t=10 \sim 20$: B 运行完成

$t=20 \sim 30$: C 运行完成

$t=30 \sim 120$: A 继续运行完成

周转时间 :

$A = 120 - 0 = 120$

$B = 20 - 10 = 10$

$C = 30 - 10 = 20$

平均 = $(120 + 10 + 20) / 3 = 50$

这比非抢占 SJF 那种情况好很多。slide 里也写到 STCF 是在 SJF 上增加抢占功能，剩余时间最短的任务优先。

但是 STCF 也有问题：

1. 仍然需要知道剩余运行时间
2. 对长任务不公平
3. 如果短任务不断到达，长任务可能饥饿
4. response time 对某些任务可能很差

2.4 Round Robin：轮转调度

Round Robin，简称 RR。

规则是：

每个 runnable 任务轮流获得一个时间片，用完时间片就切换到下一个。

例如有 A、B、C，每个时间片 10ms：

```
A 跑 10ms
B 跑 10ms
C 跑 10ms
A 再跑 10ms
B 再跑 10ms
C 再跑 10ms
...
```

它的核心优点：

```
response time 小
比较公平
适合交互式系统
```

因为新来的任务不会等一个长任务完整跑完，只要轮到它，就能得到响应。

但它也有缺点：

context switch 开销增加
turnaround time 可能变差

如果时间片太小：

切换非常频繁
context switch 开销很大

如果时间片太大：

接近 FIFO
response time 变差

所以 RR 的本质是一个 tradeoff：

小时间片：响应快，但切换开销大
大时间片：切换开销小，但响应慢

slide 里也直接说：增大时间片能减小开销比例，但也会增大 response time。偏重公平和 response time 时用 RR；不看重公平、偏重 turnaround time 时用 SJF/STCF。

3. I/O 对调度的影响

现实中的任务不只是纯 CPU 计算。很多任务会：

运行一会儿 CPU
发起 I/O
阻塞等待磁盘/网络
I/O 完成后再继续运行

slide 里强调：I/O 操作远比 CPU 周期慢，进程执行 I/O 时要 block，直到 I/O 完成。

比如任务 A：

CPU 运行 10ms
I/O 10ms
CPU 运行 10ms
I/O 10ms
...

任务 B 是纯 CPU-bound :

一直需要 CPU

一个好调度器应该在 A 等 I/O 的时候调度 B。这样 CPU 不会空闲。

所以理解 I/O 的关键是 :

进程等待 I/O 时, 它不是 `runnable`
不能被调度上 CPU
调度器应该选择其他 `runnable` 进程运行

这也解释了为什么交互式 I/O 密集型任务在很多调度器里会被优待 : 它们经常主动让出 CPU , 对 CPU 占用少 , 但对响应时间要求高。

4. MLFQ : 多级反馈队列 , 重点中的重点

这一部分是整份 slide 的核心之一。

前面的 SJF/STCF 有个共同问题 :

它们假设我们知道任务运行时间 , 但现实中通常不知道。

MLFQ 的目标就是 :

在不知道任务长度的情况下 , 尽量模拟“短任务优先” , 同时保持较好的 `response time`。

MLFQ 全称 :

Multi-Level Feedback Queue

也就是多级反馈队列。

4.1 MLFQ 的基本结构

MLFQ 有多个优先级队列：

Q0：最高优先级

Q1

Q2

Q3：最低优先级

规则 1：

如果 $\text{Priority}(A) > \text{Priority}(B)$ ，运行 A，不运行 B

规则 2：

如果 $\text{Priority}(A) = \text{Priority}(B)$ ，A 和 B 使用 Round Robin

也就是说：

先看优先级

同优先级内部再轮转

slide 中也正是这样定义 MLFQ 前两条规则。

4.2 MLFQ 的 workload 分析

MLFQ 背后的洞察是：任务大致可以分成两类。

第一类：交互式短任务 / I/O 密集型任务。

特点：

经常运行很短时间
很快因为 I/O 或用户交互而让出 CPU
对 response time 要求高

例如：

shell
编辑器
GUI
网络服务器处理小请求

第二类：长时间 CPU-bound 任务。

特点：

一直占用 CPU
不怎么主动让出 CPU
对 response time 不太敏感
更关心整体完成时间和吞吐

例如：

编译大型项目
视频编码
矩阵计算
训练模型

slide 里也说，interactive short-running 任务对 response time 要求高；longer-running CPU-bound 任务对 response time 要求不高，但对 turnaround time 更重要。

MLFQ 的漂亮之处是：它不提前知道任务类型，而是通过任务行为来猜。

4.3 MLFQ 初始版本规则

初始规则：

Rule 3:

新任务进入系统时，放到最高优先级队列。

Rule 4a:

如果一个任务用完整个时间片，说明它可能是 CPU-bound，
于是降低它的优先级。

Rule 4b:

如果一个任务在时间片用完前主动让出 CPU，
说明它可能是交互式/I/O 密集型任务，
于是保持当前优先级。

slide 里正是这样描述 Attempt #1 的优先级变化规则。

直觉是：

短任务/交互任务：

很快结束或阻塞，不会用完整个时间片

留在高优先级

响应很快

长任务/CPU-bound：

每次都把时间片用完

逐渐降低优先级

避免长期压制交互任务

所以 MLFQ 在效果上近似于：

不知道任务长度，但动态猜测谁是短任务

它相当于一种经验版 SJF。

4.4 MLFQ 为什么能照顾 I/O 密集型任务？

I/O 密集型任务经常这样：

运行一点点 CPU
发起 I/O
阻塞
I/O 完成后再回来

因为它经常在时间片用完前就让出 CPU，所以根据 Rule 4b：

它一直留在高优先级

于是 I/O 完成后，它能很快再次被调度。

这就降低了交互式任务的 response time。

slide 也明确写到：I/O 密集型任务由于 I/O 阻塞，会提前释放 CPU，所以一直留在高优先级。

5. MLFQ 的三个问题

MLFQ 初始版本看起来很美，但有三个经典问题。

5.1 问题一：饥饿 starvation

如果系统里不断有高优先级交互式任务，那么低优先级 CPU-bound 任务可能一直得不到运行。

例如：

低优先级队列里有一个长任务 A
高优先级队列不断来短任务 B、C、D、E...
调度器永远先运行高优先级任务
A 长期无法运行

这就是饥饿。

slide 里也说，如果交互式任务数量较多，低优先级 CPU-bound 任务可能饥饿，而调度算法必须避免这种灾难。

这个思想非常重要：

系统设计首先不是追求最优，而是避免灾难。

slide 里有一句很好的话：

Not pursuing the best,
But avoiding disaster

也就是：不追求理论最优，而是避免系统灾难。

5.2 问题二：用户可以作弊

初始 MLFQ 的 Rule 4b 是：

如果时间片用完前主动让出 CPU，就保持高优先级

那用户可以写一个程序：

每次快用完时间片之前
故意做一次无意义 I/O
主动让出 CPU
然后继续回来

这样它看起来像交互式任务，但实际上占用了大量 CPU。

slide 里称这是一种 Client 和 Server 之间的欺诈、对抗。用户可以在时间片结束前进行一次无意义 I/O，从而一直停留在最高优先级。

这就是系统设计里很有意思的点：

一旦规则公开，用户就可能针对规则优化甚至作弊。

5.3 问题三：workload 模式会变化

一个程序可能一开始是 CPU-bound：

大量计算

后来变成交互式：

等待用户输入

频繁 I/O

但如果它之前已经被降到低优先级，它可能没有机会上升。

所以 MLFQ 需要给任务“重新做人”的机会。

6. MLFQ 的改进

6.1 Priority Boost：周期性优先级提升

规则 5：

After some time period S , move all the jobs in the system to the topmost queue.

也就是：

每隔一段时间 S ，把系统中所有任务都移回最高优先级队列。

这解决两个问题：

1. 防止低优先级任务长期饥饿
2. 允许 workload 改变的任务重新获得高优先级

slide 里也说，Priority Boost 解决了饥饿和 workload 模式改变两个问题。

但是这里有一个参数难题：

S 设多大？

如果 S 太大：

boost 太少，长任务还是可能饥饿

如果 S 太小：

大家频繁回到最高优先级

MLFQ 退化，交互任务优势变弱

slide 里把这种难调的魔法参数叫做：

voo-doo constants

并引用 Ousterhout's Law：设计算法时尽量避免引入这种玄学常数。

6.2 Better Accounting：更好的 CPU 使用量统计

Priority Boost 还没有解决用户作弊问题。

初始规则是：

只要没用完整个连续时间片，就不降级

这太容易被钻空子。

改进后的 Rule 4 是：

Once a job uses up its time allotment at a given level,
regardless of how many times it has given up the CPU,
its priority is reduced.

翻译成人话：

不管你中间主动让出 CPU 多少次，只要你在某个优先级累计用完了该层的 CPU 配额，就降级。

比如某一层给你 20ms CPU allotment：

你一次用 19ms，然后 I/O
回来又用 1ms
累计达到 20ms
降级

这样作弊就没用了。

slide 里也明确说：占用 CPU 总数达到一定量就下降优先级，比“提前让出时间片就保留优先级”更合理，不容易被骗。

6.3 MLFQ 最终规则总结

最终 MLFQ 可以记成五条：

Rule 1:

高优先级任务优先运行。

Rule 2:

同优先级任务之间使用 Round Robin。

Rule 3:

新任务进入系统时，放入最高优先级队列。

Rule 4:

任务在某个优先级累计用完 CPU allotment 后，降低一级优先级。

Rule 5:

每隔一段时间 S，把所有任务提升到最高优先级。

这五条正是 slide 的 MLFQ Summary。

你可以这样理解 MLFQ：

它用“过去的行为”预测“未来的任务类型”：老是用满 CPU 的，当作长任务；经常主动让出 CPU 的，当作交互任务。

7. Proportional Share : 按比例公平分配

前面的算法主要是在优化：

turnaround time
response time

但 proportional share scheduler 不追求这两个指标，而是追求：

每个任务得到一定比例的 CPU 时间。

slide 里也说，它又叫 fair-share scheduler，不追求优化 turnaround time 或 response time，而是保障每个任务得到一定比例 CPU 时间。

7.1 Lottery Scheduling : 彩票调度

Lottery scheduling 的思想很有趣：

每个任务持有一定数量 tickets
每次调度时随机抽一张 ticket
抽到谁，谁运行

比如：

A 有 75 张票
B 有 25 张票

那么长期来看：

A 获得约 75% CPU
B 获得约 25% CPU

ticket 数量越多，CPU 份额越高。

slide 里也用 A 75 tickets、B 25 tickets 作为例子。

这个算法的重点不是每一次都精确公平，而是：

长期统计上公平

短期可能有波动，长期会接近比例。

7.2 Fairness 如何衡量？

slide 里提到一个 unfairness 指数 U ：

$$U = \text{第一个任务完成时间} / \text{第二个任务完成时间}$$

如果两个任务几乎同时到达，在完全公平情况下，它们应该差不多同时完成：

$$U \text{ 接近 } 1$$

如果一个任务很早完成，另一个很晚完成：

$$U \text{ 远小于 } 1$$

例如：

第一个任务 $t=10$ 完成

第二个任务 $t=20$ 完成

$$U = 10 / 20 = 0.5$$

越接近 1，越公平。

7.3 Stride Scheduling：确定性公平调度

Stride scheduling 是另一种按比例公平的方法。

大意是：

每个任务有一个 stride
每次选择当前 pass 值最小的任务运行
运行后 $\text{pass} += \text{stride}$

stride 越小，说明它应该更频繁运行。

如果 A、B、C 的 stride 分别是：

A = 100
B = 200
C = 40

那么它们获得 CPU 的比例大致是 stride 的倒数：

A : B : C = $1/100$: $1/200$: $1/40$ = 2 : 1 : 5

slide 里也给了这个例子，结果 A 调度 2 次，B 1 次，C 5 次，与预期比例一致。

7.4 为什么彩票调度有吸引力？

slide 给的理由是：

stride scheduling 需要维护 global state
新 job 加入时，起始 pass 值设多少是问题
lottery scheduling 不需要维护复杂 global state
每次随机抽票即可

随机算法的好处是：

简单
轻量
性能好
避免某些特殊 workload 下的极端坏情况

slide 也提到随机算法能避免一些特殊情况问题，例如 LRU 在 cyclic-sequential workloads 下命中率低。

你可以把 lottery scheduling 理解成：

用随机性换简单性，用长期概率保证公平。

8. 多核 CPU 调度

多核之后，调度问题不再只是“选哪个进程”，还多了一个问题：

选哪个 CPU 核来运行它？

这会引出 cache 一致性、cache affinity、负载均衡等问题。

8.1 多核新问题：cache 一致性

假设有两个 CPU 核：

Core 1 cache 了变量 x

Core 2 也 cache 了变量 x

如果 Core 1 修改了 x，那么 Core 2 cache 里的 x 就可能过时。

这就是 cache coherence 问题。

slide 里也说，多核 CPU 的新问题是 cache 一致性：核 1 和核 2 都 cache 同一份数据，核 1 修改后，核 2 读数据可能不一致。解决方式之一是 bus snooping，但会增加性能开销。

8.2 MESI 协议

slide 里讲了 MESI，这是 cache line 的四种状态：

M: Modified
E: Exclusive
S: Shared
I: Invalid

简单理解：

M : Modified

这个 cache line 被当前核心修改过
它是系统中唯一正确版本
其他核心的该 cache line 无效

E : Exclusive

只有当前核心 cache 了这个 line
内容和内存一致
写的时候可以直接变成 M

S : Shared

多个核心都 cache 了这个 line
内容和内存一致
如果某个核心要写，需要让其他副本失效

I : Invalid

这个 cache line 无效
当作没有

slide 里给出的 MESI 描述也是围绕这四种状态。

8.3 多线程原子性与锁

多核环境下，如果多个线程同时操作共享数据，就会有原子性问题。

比如一个链表：

- 两个线程同时看到 head
- 两个线程都删除 head
- 可能导致 double free

解决方式通常是加锁。

但锁会带来性能开销：

- 锁竞争
- cache line bouncing
- 线程被调度到越多不同核心，冲突越复杂

slide 里也说，多线程原子性问题可以加锁，但会带来性能开销；线程可能被调度到的核越多，冲突越大，性能越差。

8.4 Cache Affinity：缓存亲和性

Cache affinity 是多核调度的重点。

意思是：

一个进程如果连续在同一个 CPU 核上运行，它之前用过的数据和 TLB 项可能还在这个核的 cache/TLB 里，因此性能更好。

如果进程频繁被迁移到不同 CPU 核：

- 旧核心上的 cache/TLB 资源浪费
- 新核心上需要重新填 cache/TLB
- 性能下降

slide 里明确说，一个进程连续调度到同一个 CPU 核上，cache 和 TLB 都有效，性能会更好；调度到不同核则 cache 资源浪费。

所以多核调度有一个矛盾：

负载均衡希望进程能迁移

cache affinity 希望进程少迁移

这又是一个 tradeoff。

9. 多核调度的两种基本方案

9.1 SQMS : 单队列多处理器调度

SQMS :

Single-Queue Multiprocessor Scheduling

意思是 :

所有 CPU 核共享一个全局 run queue

哪个核空了, 就从这个队列拿一个任务

优点 :

负载均衡好

实现逻辑直观

不会轻易出现某个核空闲、另一个核排队很多的情况

缺点 :

全局队列需要加锁

多个核心同时访问队列, lock contention 大

cache affinity 差

进程可能被不同核心拿走

slide 里也说 SQMS 的优点是 load balance , 缺点是 lock 开销大、cache affinity 差、性能差。

可以改进为 :

尽量把任务调度到同一个 CPU 核
只在少数情况下迁移任务

但这种改进扩展性仍然不好。

9.2 MQMS：多队列多处理器调度

MQMS：

Multi-Queue Multiprocessor Scheduling

意思是：

每个 CPU 核有自己的 run queue
每个核心主要从自己的队列里取任务

优点：

锁竞争少
cache affinity 好
扩展性强

因为每个核操作自己的队列，不需要大家抢一个全局队列。

缺点：

load imbalance 负载不均衡

比如：

CPU 0 队列里有很多长任务
CPU 1 队列里任务很快完成
CPU 1 空闲
CPU 0 忙死

这就浪费 CPU。

slide 里也说 MQMS 的优点是 lock 冲突少、cache affinity 好；缺点是 load balance，需要 work stealing。

9.3 Work Stealing：工作窃取

为了解决 MQMS 的负载不均衡，可以用 work stealing：

如果某个 CPU 核自己的队列空了
它就去别的核心队列里偷一两个任务过来

这样可以改善负载均衡。

但是偷得太频繁也不好：

增加锁开销
破坏 cache affinity
迁移成本上升

所以 work stealing 的频率也需要权衡。

slide 里也说，不饱和队列可以从其他队列拿一两个 job，但频率提高也会有问题。

10. Linux 调度器对应关系

slide 里提到 Linux 里没有一种确定方案完全优于其他。

例子包括：

CFS: Completely Fair Scheduler，多队列，类似 stride scheduler
BFS: BF Scheduler，单队列
O(1) Scheduler：多队列，类似 MLFQ

openEuler 的 CFS 也引入虚拟运行时间，按照系统负载和普通进程优先级给进程分配 CPU 使用时间比例，保证相对公平。

CFS 的核心可以先这样记：

谁“虚拟运行时间 vruntime”最小，说明谁相对没怎么用 CPU，就优先调度谁。

它不是简单 RR，也不是简单 SJF，而是在公平性和效率之间折中。

11. 这份 slide 最应该掌握的重点

我给你压缩成考试版。

第一组：基本指标

必须会区分：

`turnaround time = completion time - arrival time`

`response time = first run time - arrival time`

`waiting time = turnaround time - running time`

其中：

`turnaround time` 看任务多久完成

`response time` 看多久第一次响应

`fairness` 看 CPU 分配是否公平

第二组：基本算法

FIFO / FCFS:

先来先服务

简单，但长任务在前会拖累短任务

SJF:

最短作业优先

所有任务同时到达且已知长度时，平均 turnaround time 最优

但不知道任务长度，且非抢占时处理不了后来短任务

STCF / PSJF:

剩余时间最短优先

抢占式 SJF

能改善后来短任务的问题

但仍需知道剩余时间，且可能饿死长任务

RR:

轮转调度

response time 好，公平性较好

但 context switch 开销变大

时间片大小需要权衡

第三组：MLFQ

MLFQ 是重点，记住：

目标：

不知道任务长度时，近似实现短任务优先，同时保持交互响应。

基本机制：

多个优先级队列。

高优先级先运行。

同优先级 RR。

新任务进最高优先级。

用满 CPU 配额就降级。

周期性 priority boost 防止饥饿。

MLFQ 最重要的思想：

根据任务过去的行为来猜它未来的类型。

具体说：

老是用完整个 CPU 时间片 => CPU-bound => 降级

经常提前让出 CPU => interactive/I/O-bound => 保持高优先级

改进点：

Priority Boost:

解决饥饿和 workload 改变问题。

Better Accounting:

防止用户通过假 I/O 欺骗调度器。

第四组：Proportional Share

目标：

不追求最小 turnaround/response，而是按比例分 CPU。

Lottery Scheduling:

ticket 越多，被调度概率越大。

长期来看 CPU 份额接近 ticket 比例。

Stride Scheduling:

确定性比例调度。

选择 pass 值最小的任务。

运行后 $pass += stride$ 。

stride 越小，获得 CPU 越多。

第五组：多核调度

多核调度的核心矛盾：

load balance vs cache affinity

两种基本方案：

SQMS：

一个全局队列。

优点：负载均衡好。

缺点：锁开销大，cache affinity 差。

MQMS：

每个 CPU 一个队列。

优点：锁冲突少，cache affinity 好。

缺点：容易负载不均衡。

解决：work stealing。

多核还要理解：

cache coherence

MESI

锁竞争

进程迁移成本

12. 最后一口气版总结

CPU 调度本质上是在回答：

现在该让哪个 runnable 任务上 CPU？

如果是多核，还要回答：让它在哪个核上跑？

不同算法站在不同价值观上：

FIFO:

最简单，谁先来谁先跑。

SJF/STCF:

追求平均完成时间短，但可能不公平。

RR:

追求响应快和公平，但牺牲一些效率。

MLFQ:

不知道任务长度时，用历史行为动态调整优先级，兼顾短任务响应和长任务推进。

Lottery/Stride:

不追求最短完成时间，而是保证按比例公平分 CPU。

多核调度:

要在负载均衡、锁开销、cache affinity 之间权衡。

最核心的系统哲学是这句：

调度没有银弹。你优化 turnaround time，可能伤害 fairness；你优化 response time，可能增加 context switch；你优化 load balance，可能破坏 cache affinity。系统设计往往不是追求某个指标绝对最优，而是在灾难边缘做漂亮的折中。